ThreatScope

Named-Entity Recognition for Cyber-Threat-Intelligence Reports

An on-prem NER microservice that extracts threat entities — APT groups, malware, tools, IPs, CVEs — from unstructured CTI text.

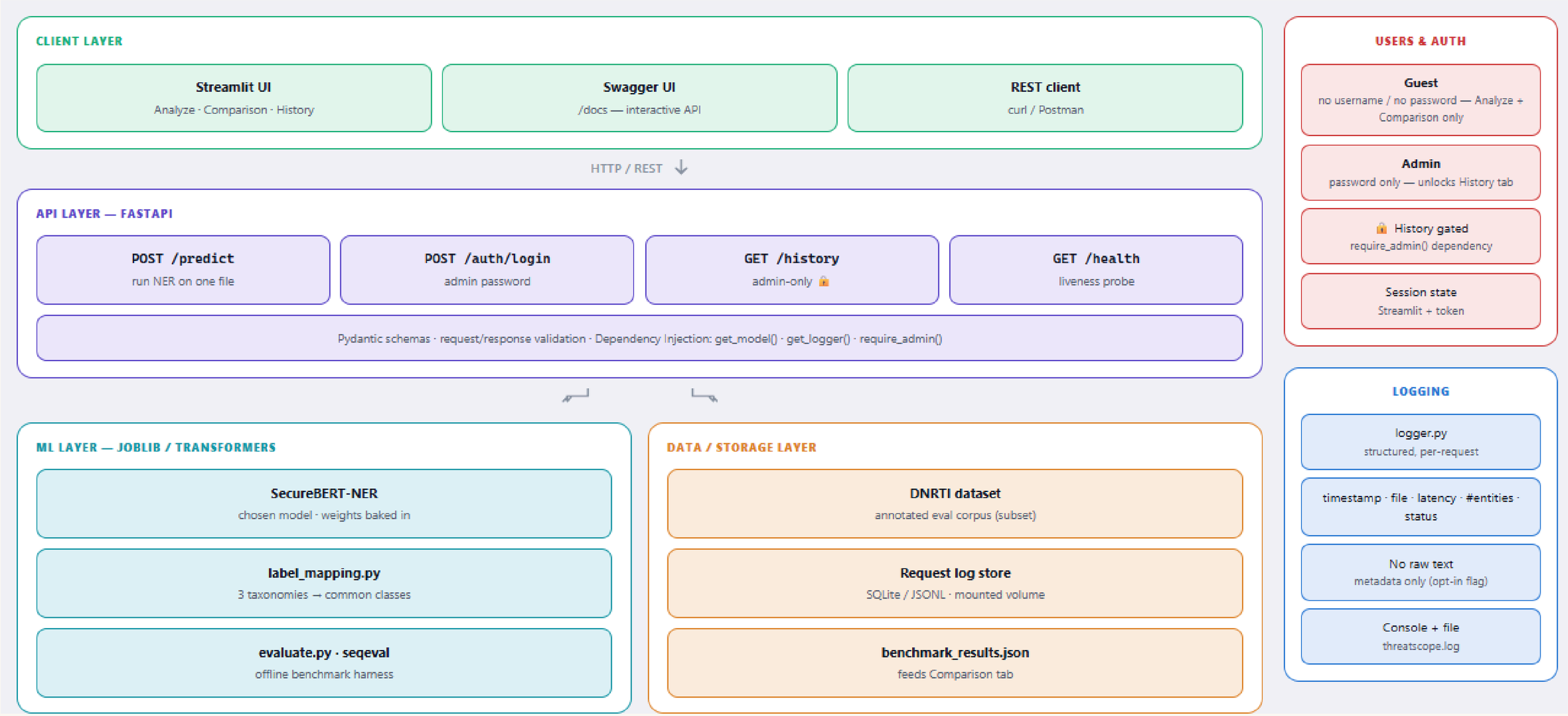
SAMPLE EXTRACTION

APT28 deployed X-Agent via CVE-2023-23397 , beaconing to 185.220.101.47 .

APT-GROUP MALWARE CVE IP

Elad Moshe · July 2026

Product Architecture — RUNTIME



MODEL SELECTION

Model Benchmarking

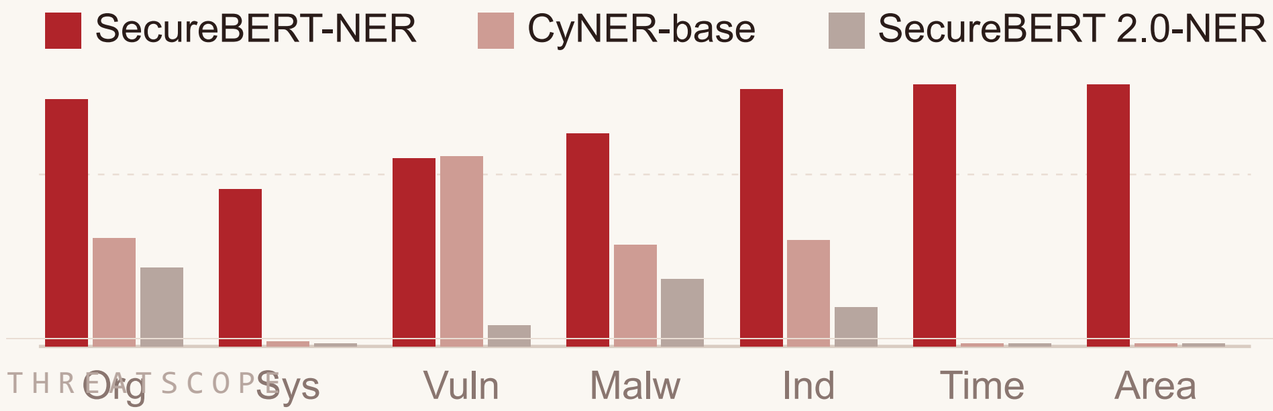
Overall span-level F1 · `seqeval` · DNRTI test

SecureBERT-NER WINNER — **0.70**

CyNER (xlm-roberta-base) — 0.27

SecureBERT 2.0-NER — 0.14

Per-class F1 — winner leads on every class except Vulnerability (tie)



PROTOCOL

- One self-contained Jupyter notebook — runs on Colab or locally
- DNRTI dataset · CoNLL BIO format · 662 test sentences
- Entity-/span-level eval: `seqeval` + `nervaluate` (strict / exact / partial / type)
- Micro / macro / weighted F1 with per-class support

Excluded: CyNER-large (`cyner-xlm-roberta-large`) — the HF repo ships config but no public weights.

HONEST CAVEAT

SecureBERT-NER was trained on APTNER, whose taxonomy closely matches DNRTI — the win is partly domain alignment. SecureBERT 2.0 collapses via over-prediction.

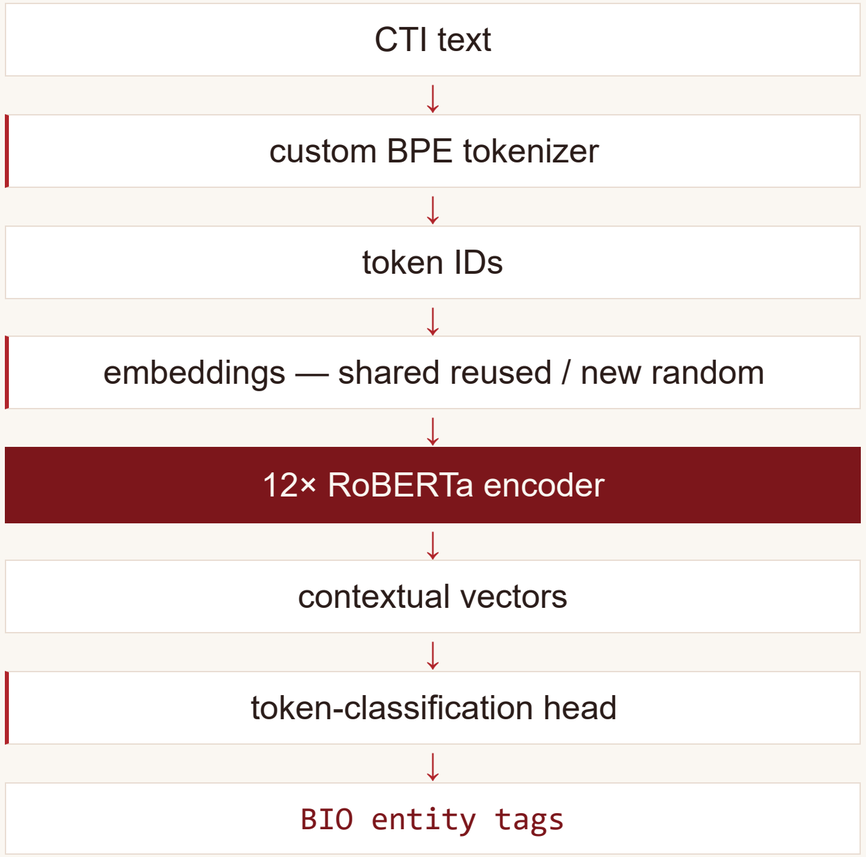
CHOSEN MODEL

The SecureBERT Backbone

Not a new architecture. RoBERTa-base, unchanged — 12 encoder layers, 12 heads, hidden 768, ~50k vocab, 512 max tokens.

1. **Custom BPE tokenizer trained on cybersecurity text** — domain terms (*ransomware, honeypot, Metasploit*) become single tokens instead of fragmented subwords.
2. **Vocabulary-aligned embedding init** — shared tokens keep their original RoBERTa IDs (preserving pretrained knowledge) + small Gaussian perturbation; truly-new tokens get Xavier init.
3. **Continued MLM pretraining** on a large cybersecurity corpus.

Same proven encoder; domain-adapted tokenizer, vocabulary and pretraining data — the SciBERT / BioBERT recipe applied to security. SecureBERT-NER = this backbone fine-tuned for token classification on APTNER (21 native entity classes).



Inference Post-Processing

Raw pipeline output is noisy subword spans — this wrapper turns it into clean, deduplicated, offset-accurate entities for the API and UI.

01 **Load once** — model cached with `@lru_cache`, warmed up at startup.

02 **Chunk** long documents at whitespace to stay under the 512-token limit, tracking character offsets back to the original text.

03 **Infer** with `aggregation_strategy="none"` — keep raw per-token predictions.

04 **Merge** fragmented same-class subword tokens into whole entities, using character offsets on the original string.

05 **Normalise** — strip BIO prefixes, group by native SecureBERT class, de-duplicate, keep display order.

RAW MODEL OUTPUT

`B-IP · 185` `I-IP · .220` `I-IP · .101` `I-IP · .47`

↓

MERGED ENTITY

```
{ "text": "185.220.101.47",
  "label": "IP", "start": 61,
  "end": 75, "score": 0.97 }
```

Dockerization

IMAGE HYGIENE

- Separate images for API and UI
- CPU-only PyTorch wheels — slim image, no CUDA payload
- Non-root user + `HEALTHCHECK`
- Weights `COPY`-baked into the image — no network at runtime
- `TRANSFORMERS_OFFLINE` · `HF_HUB_OFFLINE`
- Logs on a bind-mounted volume — persist across restarts

WHY TWO CONTAINERS

Separation of concerns — UI and inference scale, deploy, and fail independently.

Independent scaling — replicate the stateless API under load without touching the UI.

Smaller blast radius — the model container needs no public exposure; only the UI is user-facing.

Faster iteration — change the front end without rebuilding the heavy ML image.

Clean interface — a documented HTTP contract; either side can be swapped with no code change.

DEFENCE-IN-DEPTH

Input Hardening & XSS Safety

Layered validation → safe decoding → safe rendering, on every untrusted upload.

LAYER 1

Input validation

`validate_and_decode()`

Rejects non-.txt by extension; enforces a max file size

Rejects empty / whitespace-only files

Decodes UTF-8 with `errors="replace"`

Strips control characters (except tab / newline / CR) — neutralises malformed and binary input

LAYER 2

Output encoding

`html.escape()`

The UI renders extracted entities as highlighted HTML

All model output and source text is HTML-escaped before rendering

User- or model-controlled strings can never inject markup or script into the page

LAYER 3

Privacy-minimal logging

`request log · SQLite`

Stores request metadata only — never document contents

A compromised or subpoenaed log leaks no CTI source material

ROADMAP

Future Work

Analytics dashboard

Entity-frequency trends, per-class volumes, drift monitoring.

Multi-tenant async architecture

Async inference workers + a queue, per-tenant isolation, horizontal autoscaling for concurrent load.

User management & auth

OAuth2 / JWT bearer tokens, per-tenant API keys.

More input formats

PDF, HTML, DOCX, STIX / MISP feeds.

Integration

Plug into the organization's existing apps and/or ship as a browser add-on.

Better model performance

Evaluate additional models — including larger LLMs and alternative approaches.

HONEST ASSESSMENT

Limitations

01 **Model performance ceiling** — the best model reaches only ~0.70 span-level F1; not production-grade for fully automated extraction without a human in the loop.

02 **Subword fragmentation** — multi-token entities (IPs, hashes, long tool names) get fragmented and occasionally mis-merged; boundary detection is imperfect.

03 **Precision / recall trade-off** — false negatives and spurious false positives, uneven across classes; rare classes have low support and weak recall.

04 **Domain dependence** — evaluated on DNRTI; the winner is domain-aligned with its APTNER training data, so generalisation to other CTI styles is unverified.

05 **Plain-text input only** — no support for file types other than .txt (PDF, HTML, DOCX); CPU-latency-bound on long documents.